

SmartLogix

Documentacion tecnica - Evaluacion Parcial 3

Integracion de arquitectura de microservicios

Este documento presenta la propuesta de arquitectura de microservicios del proyecto SmartLogix, explicando la integracion entre frontend, backend, API REST, persistencia de datos, pruebas unitarias y versionamiento en GitHub.

Dato	Informacion
Proyecto	SmartLogix
Evaluacion	Evaluacion Parcial 3
Arquitectura	Microservicios + API REST + Docker Compose
Frontend	React/Vite servido con Nginx
Backend	Spring Boot, Spring Cloud Gateway, Eureka y servicios REST
Persistencia	JPA/Hibernate con bases por microservicio y volumen Docker
Repositorio principal	https://github.com/shelsye/smartlogix-platform.git
Repositorio frontend	https://github.com/shelsye/smartlogix-frontend.git
Repositorio backend	https://github.com/shelsye/smartlogix-backend-services.git

Indice

1. Objetivo del documento
2. Resumen del sistema SmartLogix
3. Arquitectura de microservicios
4. Diagrama de arquitectura
5. Componentes desarrollados
6. Comunicacion mediante API REST
7. Persistencia de datos
8. Pruebas unitarias y metricas
9. Docker Compose y ejecucion
10. Versionamiento en GitHub
11. Evidencias para defensa
12. Conclusiones

1. Objetivo del documento

El objetivo de esta documentacion es respaldar la Evaluacion Parcial 3 del proyecto SmartLogix, demostrando una arquitectura basada en microservicios, comunicacion entre frontend y backend mediante API REST, persistencia de datos y una estrategia de pruebas unitarias.

El documento esta preparado para ser entregado junto con el proyecto comprimido y los enlaces de GitHub solicitados por la evaluacion.

Alcance

- Presentar el esquema de arquitectura de microservicios.
- Explicar la integracion entre frontend, API Gateway y servicios backend.
- Describir la persistencia de datos mediante JPA/Hibernate.
- Indicar las pruebas unitarias necesarias y las metricas que deben reportarse.
- Documentar los repositorios GitHub del proyecto.
- Entregar un diagrama de arquitectura en formato PNG.

2. Resumen del sistema SmartLogix

SmartLogix es una plataforma de gestion logistica que integra tienda, inventario, rutas de envio, ordenes de compra, pago ficticio, boleta electronica y reportes administrativos. La solucion separa responsabilidades en microservicios para mejorar mantenimiento, escalabilidad y trazabilidad.

Funciones principales

- Registro e inicio de sesion para usuarios clientes y administrador.
- Dashboard de administrador con inventario, ordenes y reportes.
- Catalogo de productos para clientes sin exponer informacion interna de stock.
- Validacion real de stock desde backend antes de comprar.
- Adaptador inteligente de envios para las 16 regiones de Chile.
- Seleccion y persistencia de ruta aceptada.
- Cupon de bienvenida BIENVENIDA10 con 10% de descuento y uso unico.
- Pago con tarjeta ficticia y generacion de boleta electronica.
- Ejecucion completa con Docker Compose.

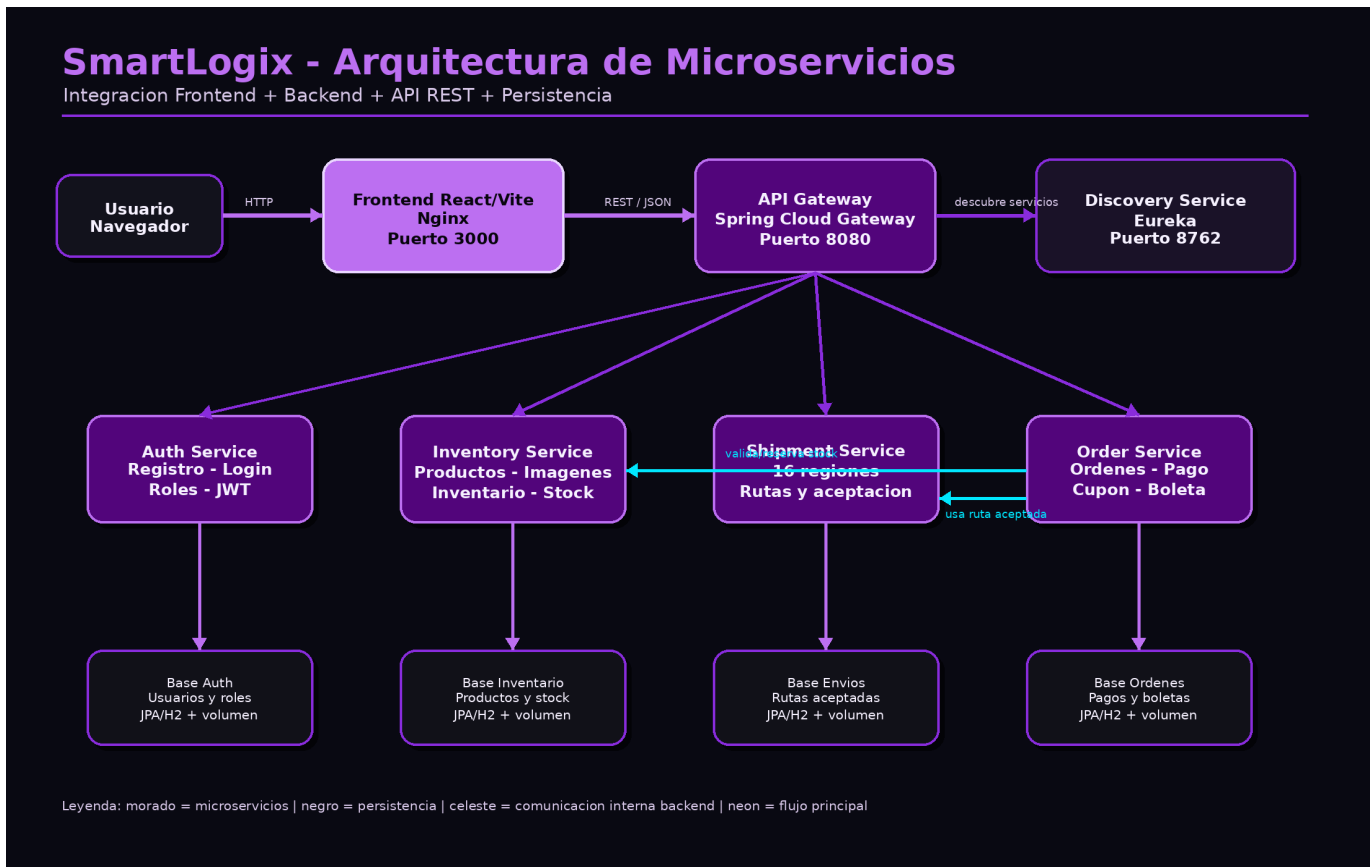
3. Arquitectura de microservicios

La arquitectura del sistema se organiza mediante un frontend React que consume una API REST centralizada en un API Gateway. El Gateway distribuye las solicitudes hacia microservicios especializados. Cada microservicio implementa sus reglas de negocio y guarda sus datos de forma independiente.

Capa	Componente	Responsabilidad
Cliente	Frontend React/Vite	Interfaz de usuario, login, dashboards, catalogo, carrito, pago y visualizacion de boleta.
Entrada API	API Gateway	Punto unico de entrada para las peticiones REST del frontend.
Descubrimiento	Discovery Service Eureka	Registro y descubrimiento de microservicios.
Seguridad	Auth Service	Registro, login, roles y autenticacion JWT.
Inventario	Inventory Service	CRUD de productos, imagenes, stock, activacion y desactivacion.
Envios	Shipment Service	Calculo de rutas, 16 regiones, tres opciones y ruta aceptada.
Ordenes	Order Service	Ordenes, pagos, cupon, boleta electronica y reportes.
Datos	Bases por servicio	Persistencia independiente con JPA/Hibernate y volumen Docker.

4. Diagrama de arquitectura

El siguiente diagrama muestra la integración entre frontend, API Gateway, microservicios y persistencia. También muestra la comunicación interna entre order-service, inventory-service y shipment-service.



El archivo PNG independiente también se entrega junto a este PDF para que pueda subirse por separado si el formato de entrega lo solicita.

5. Componentes desarrollados

Componente	Tecnologia	Descripcion
Frontend	React, Vite, Nginx	Aplicacion visual. Muestra datos, consume API REST y no debe contener la logica principal
API Gateway	Spring Cloud Gateway	Enruta solicitudes /api/auth, /api/inventory, /api/shipments y /api/orders.
Discovery Service	Eureka	Permite registrar y ubicar microservicios en la red Docker.
Auth Service	Spring Boot + JPA	Gestiona usuarios, login, administrador, clientes y JWT.
Inventory Service	Spring Boot + JPA	Gestiona productos, imagenes, precios, stock y estados.
Shipment Service	Spring Boot + JPA	Gestiona regiones, rutas, transportistas, costos y ruta aceptada.
Order Service	Spring Boot + JPA	Gestiona ordenes, detalles, descuentos, pagos, boletas y reportes.

Responsabilidades clave

- El frontend solo presenta la informacion y ejecuta llamadas HTTP.
- El backend valida reglas de negocio, como stock, descuentos, rutas y pagos.
- La base de datos es la fuente de verdad del sistema.
- Los cambios deben persistir despues de recargar la pagina o reiniciar contenedores, siempre que no se eliminen los volúmenes Docker.

6. Comunicacion mediante API REST

La comunicacion entre componentes se realiza mediante API REST usando JSON. El frontend consume el API Gateway y este redirige hacia el microservicio correspondiente.

Funcion	Metodo	Endpoint	Respuesta esperada
Registro usuario	POST	/api/auth/register	HTTP 200/201
Login	POST	/api/auth/login	HTTP 200 + token JWT
Catalogo cliente	GET	/api/inventory/catalog	HTTP 200
Crear producto	POST	/api/inventory	HTTP 200/201
Actualizar stock	PUT	/api/inventory/{id}/stock	HTTP 200
Calcular rutas	POST	/api/shipments/options	HTTP 200 + 3 opciones
Aceptar ruta	POST	/api/shipments/selections	HTTP 200
Crear orden	POST	/api/orders	HTTP 200/201
Pagar orden	POST	/api/orders/{id}/pay	HTTP 200 + boleta
Ver boleta	GET	/api/orders/{id}/receipt	HTTP 200

Ejemplo: aceptar ruta

```
POST /api/shipments/selections Authorization: Bearer <TOKEN_CLIENTE> Content-Type: application/json { "region": "Metropolitana de Santiago", "optionType": "ECONOMICO", "routeCode": "RM-ECO-001" } Respuesta esperada: HTTP 200 OK
```

7. Persistencia de datos

La persistencia se implementa desde los microservicios backend mediante JPA/Hibernate. El frontend no es la fuente de verdad del sistema; cada cambio importante debe viajar por el flujo Frontend -> API Gateway -> Microservicio -> Base de datos.

Servicio	Datos persistidos	Mecanismo
auth-service	Usuarios, roles, perfil y credenciales cifradas.	Entidades JPA + repositorios Spring Data.
inventory-service	Productos, categorias, precios, imagenes, stock, estado activo/inactivo.	Entidades JPA + repositorios Spring Data.
shipment-service	Regiones, alternativas de envio y ruta aceptada por usuario/orden.	Entidades JPA + repositorios Spring Data.
order-service	Ordenes, detalles, pago ficticio, cupon aplicado, boleta y reportes.	Entidades JPA + repositorios Spring Data.

Garantias de persistencia

- Los metodos save() y operaciones CRUD se ejecutan en el backend.
- El stock se actualiza desde inventory-service y se mantiene al recargar.
- La ruta aceptada se guarda en shipment-service y no se recalcula al pagar.
- La orden y la boleta se guardan en order-service.
- Los datos se conservan en volumen Docker mientras no se ejecute docker compose down -v.

Flujo de compra persistente

Cliente selecciona producto -> acepta ruta -> backend guarda ruta -> cliente paga -> order-service valida stock y ruta -> se registra orden -> se genera boleta electronica -> el cliente puede recargar y seguir viendo sus datos.

8. Pruebas unitarias y metricas

Las pruebas unitarias permiten validar que cada componente responda correctamente. Como los resultados reales deben obtenerse al ejecutar el proyecto, este documento deja indicados los casos y metricas que el equipo debe completar con evidencia.

Componente	Prueba sugerida	Resultado esperado	Evidencia
auth-service	Registro de cliente	Usuario guardado y HTTP 200/201	Network o test unitario
auth-service	Login administrador	JWT valido y rol ADMIN	Respuesta API
inventory-service	Actualizar stock	Stock persistido y HTTP 200	Network + recarga
inventory-service	Producto sin imagen	Imagen por defecto visible	Captura frontend
shipment-service	Calcular rutas	3 opciones para region	Respuesta API
shipment-service	Aceptar ruta	Ruta persistida y HTTP 200	Network + recarga
order-service	Crear orden con stock	Orden guardada	Respuesta API
order-service	Compra sin stock	Compra rechazada	Respuesta error controlado
order-service	Cupon BIENVENIDA10	10% una vez por usuario	Orden con descuento
order-service	Pago ficticio	Boleta generada	Captura boleta
frontend	npm run build	Build exitoso	Terminal

Metricas a reportar

Metrica	Objetivo sugerido	Resultado real
Pruebas ejecutadas	100% de pruebas definidas	Completar por el equipo
Pruebas exitosas	>= 90%	Completar por el equipo
Cobertura de lineas	>= 70% si usan JaCoCo	Completar por el equipo
Cobertura de metodos	>= 70% si usan JaCoCo	Completar por el equipo
Build frontend	Exitoso	Completar por el equipo
Docker Compose	Servicios UP	Completar por el equipo

Importante: no se deben inventar porcentajes de cobertura. Las metricas reales se completan despues de ejecutar las pruebas.

9. Docker Compose y ejecucion

El proyecto se levanta mediante Docker Compose. Cada microservicio y el frontend se ejecutan en contenedores independientes dentro de la misma red Docker.

```
docker compose down --remove-orphans docker compose up --build
```

Accesos locales

Componente	URL
Frontend	http://localhost:3000
API Gateway	http://localhost:8080
Eureka	http://localhost:8762

Comandos utiles

```
docker compose ps docker compose logs -f docker compose logs -f inventory-service docker  
compose logs -f shipment-service docker compose logs -f order-service
```

Para conservar los datos cargados, no se debe ejecutar docker compose down -v, ya que ese comando elimina los volúmenes persistentes.

10. Versionamiento en GitHub

Los componentes del proyecto se versionan en GitHub. La entrega debe incluir enlaces actualizados y accesibles.

Repositorio	URL	Proposito
Principal	https://github.com/shelsye/smartlogix-platform.git	Documentacion general, docker-compose, guias de ejecucion e integracion.
Frontend	https://github.com/shelsye/smartlogix-frontend.git	Codigo React/Vite, package.json, src, Dockerfile y nginx.conf.
Backend Services	https://github.com/shelsye/smartlogix-backend-services.git	Microservicios Spring Boot, API REST, persistencia, entidades, repositorios y config.

Comandos base

```
git status git add . git commit -m "Entrega Evaluacion Parcial 3 SmartLogix" git push
```

11. Evidencias para defensa

Para defender el proyecto en la exposicion, se recomienda presentar evidencias concretas. Las capturas deben demostrar comunicacion API, persistencia y funcionamiento real.

- docker compose ps con los servicios levantados.
- Login de administrador y dashboard.
- Edicion de stock con HTTP 200 en Network.
- Recarga de pagina mostrando que el stock permanece.
- Catalogo del cliente sin stock visible.
- Adaptador con tres opciones de ruta.
- Aceptar ruta con HTTP 200 en Network.
- Recarga mostrando que la ruta aceptada permanece.
- Pago ficticio aprobado.
- Boleta electronica generada.
- Repositorio GitHub actualizado.

Guion breve para exposicion

SmartLogix esta construido con arquitectura de microservicios. El frontend React consume la API REST mediante el API Gateway. Los microservicios separan responsabilidades: autenticacion, inventario, envios y ordenes. Cada servicio guarda sus datos con JPA/Hibernate. Docker Compose permite levantar toda la solucion de forma integrada. Las pruebas y evidencias demuestran que los cambios se guardan en backend y permanecen despues de recargar.

12. Conclusiones

La propuesta de SmartLogix cumple con los objetivos de la Evaluacion Parcial 3 porque integra frontend, backend, API REST, persistencia y versionamiento en GitHub bajo una arquitectura de microservicios.

La separacion por servicios permite mantener responsabilidades claras: autenticacion, inventario, envios y ordenes. La persistencia se garantiza desde backend mediante JPA/Hibernate y volúmenes Docker. El uso de API Gateway y Eureka facilita la comunicacion y organizacion de servicios.

Las pruebas y evidencias deben mostrar que el sistema responde con HTTP 200 en operaciones exitosas, que los datos permanecen al recargar y que el proyecto puede levantarse con Docker Compose.